Circuitos Lógicos

Projeto de Circuitos Lógicos

A síntese de circuitos lógicos pode ser executada em cinco níveis:

N	Nível	Atividades
1	Sistema	- especificação de requisitos - particionamento
2	Algoritmo	- especificação de comportamento - concorrência - complexidade - representação de dados
3	Arquitetura	- representação de dados - sinais e controle - paralelismo e pipelining - data paths
4	Lógico	- circuitos - otimizações em portas e transistores - mapeamento em bibliotecas
5	Físico	- otimização lógica - planejamento de layout - fabricação e encapsulamento

Codificadores e Decodificadores

Sequências de bits podem ser usadas para codificar valores numéricos. Dois códigos binários têm aplicações especiais: o **BCD** (*Binary-Coded Decimal*) e o de **Gray**.

Decodificadores em geral são circuitos lógicos capazes de ativar uma saída de acordo com uma seleção de sinais de entrada:

- Um decodificador de **nível alto** ativa uma saída quando uma das entradas estiver em nível 1 e as outras em nível 0.
- Um decodificador de **nível baixo** ativa uma saída quando uma das entradas estiver em nível 0 e as outras em nível 1

Código BCD

O código BCD é basicamente uma forma de codificar valores numéricos na base 10 em seus equivalentes binários.

```
| Decimal | BCD  |
|---------|------|
|    0    | 0000 |
|    1    | 0001 |
```

```
|    2    | 0010 |
|    3    | 0011 |
|    4    | 0100 |
|    5    | 0101 |
|    6    | 0110 |
|    7    | 0111 |
|    8    | 1000 |
|    9    | 1001 |
```

- Exemplo:

  Codificar o valor decimal 369 em código BCD: 369 = (0011) (0110) (1001).

  Um circuito codificador “decimal-para-BCD” é aquele capaz de mapear um conjunto de entradas (0-9) em um outro conjunto de quatro valores binários (s3s2s1s0), se apenas uma das entradas for acionada.

Código de Gray

O código de Gray serve para expressar sequências binárias nas quais dois valores sucessivos tenham apenas um bit de diferença (distância de *Hamming* = 1). Esse código também é conhecido com “código binário refletido“.

```
| Decimal | Binário | Gray |
|---------|---------|------|
| 0       | 000     | 000  |
| 1       | 001     | 001  |
| 2       | 010     | 011  |
| 3       | 011     | 010  |
| 4       | 100     | 110  |
| 5       | 101     | 111  |
| 6       | 110     | 101  |
| 7       | 111     | 100  |
```

Multiplexador

Multiplexadores (ou seletores de dados) são circuitos lógicos capazes de atuar como chaves digitais: recebem **várias entradas e selecionam uma delas**, em certo instante, e realizam sua transferência para a saída, mediante um código de seleção. Podem ser usados para rotear dados, sequenciar operações, realizar conversões do tipo paralelo-série e gerar tabelas ou formas de ondas.

Multiplexadores podem selecionar **mais sinais** dependendo do número de bits (tamanho) da **chave de seleção**.

```
if ( selector == 1 )
   output = input_b;
else
   output = input_a;

         selector
            _|_
input_a ---|0  \
           |    |--- output
input_b ---|___/

         selector
             |   ____
             |  |    |\      ____
             +--|  1 |-+----|    |
             |  |____|      | &  |---+   ____
input_a -----|--------------|____|   |__|    |
             |               ____     __| >=1|--- output
             +--------------|    |   |  |____|
                            | &  |---+
input_b --------------------|____|
```

Demultiplexador

Demultiplexadores (ou distribuidores de dados) são circuitos capazes de receber **um sinal de entrada e distribuí-lo em uma dentre várias saídas**, segundo um código de seleção. Podem ser usados para distribuir um mesmo sinal de ativação ou sequenciamento (*clock*) para vários circuitos.

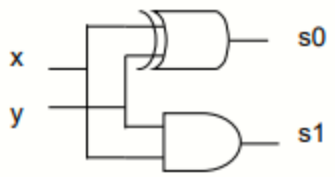
```
if ( selector == 1 )
   output_1 = input_a;
else
   output_0 = input_a;

         selector
            _|_
           /  0|--- output_0
nput_a ---|    |
           \___|--- output_1

         selector
             |   ____
             |  |    |\      ____
             +--|  1 |-+----|    |
```

```
                |  |____|      | &  |--- output_0
  input_a --+--|--------------|____|
            |  |               ____
            |  +--------------|    |
            |                 | &  |--- output_1
            +-----------------|____|
```

Aplicações Aritméticas

Somadores

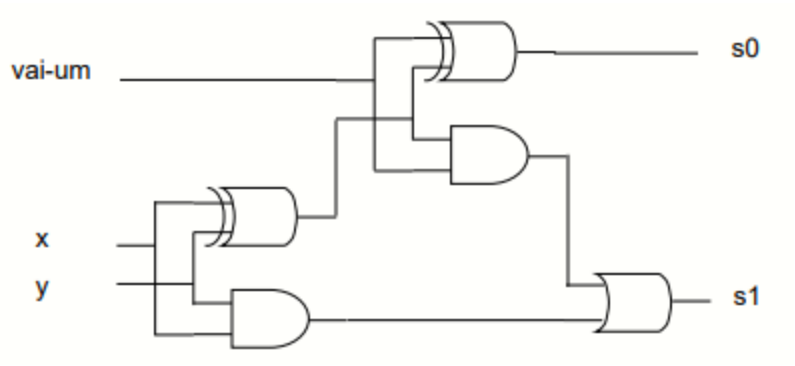
- Somador de 2 bits ("meia-soma" ou "meio-somador" ou "*half-adder*") :




```
  - Equações características :

        Forma completa (ou canônica) :

        s1 = v =     a.b     (v -> "vai-um" )
        s0 = r = a'.b + a.b' (r -> resultado)

        Forma compacta (soma de produtos) :

        s1 = v = SoP (3)
        s0 = r = SoP (1,2)

  - Verilog :

        - Definição por expressão :

              module halfAdder ( output carryOut, output sum, input a, input b
);
                  assign s0 = a ^ b; // a xor b
                  assign s1 = a & b; // a and b
              endmodule // halfAdder

        - Definição por portas lógicas ( gates ):

              module halfAdder ( output carryOut, output sum, input a, input b
);
                  xor XOR (     sum , a, b );
```

```
                    and AND ( carryOut, a, b );
                endmodule // halfAdder
```

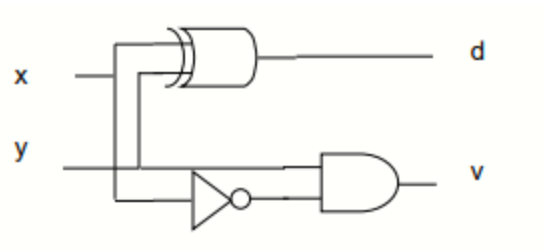
- Somador de 3 bits ("somador completo em cascata" ou "*ripple-carry adder*") :



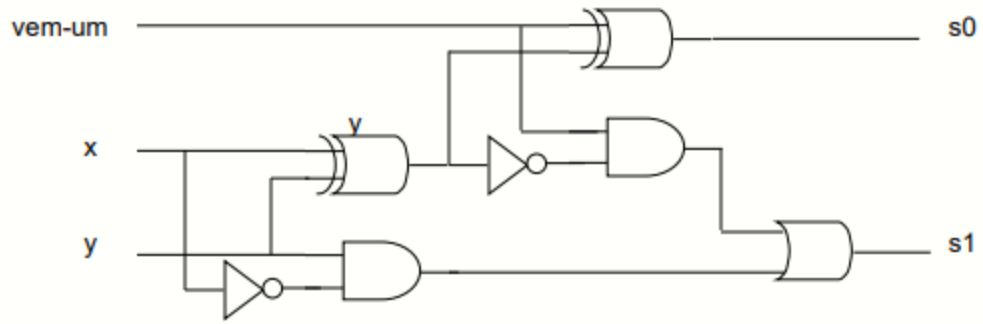

```
  - Definição por expressão :

        module fullAdder ( output carryOut, output sum, input a, input b,
input carry );
            assign carryOut = (a ^ b) & carry | (a & b);
            assign sum = (a ^ b) ^ carry;
        endmodule // fullAdder

  - Definição por portas lógicas ( gates ) :

        module fullAdder ( output carryOut, output sum, input a, input b,
input carry );
            wire w1, w2, w3;
            xor XOR_1 ( w1, a ,  b );
            xor XOR_2 ( sum, w1,  carry );
            and AND_1 ( w2, w1,  carry );
            and AND_2 ( w3, a ,  b );
            and OR__1 ( carryOut, w2, w3 );
        endmodule // fullAdder

        module fullAdder ( output carry, output sum, input a,  input b,  input
carry );
            wire w1,w2,w3;
            halfAdder HA0 ( w1, w2, a, b );
            halfAdder HA1 ( w3, sum, w2, carry );
            or        OR1 ( carry, w1, w3 );
        endmodule // fullAdder
```

Subtratores

- Subtrator de 2 bits ("meia-diferença" ou "half-difference")



- Subtrator de 3 bits ("subtrator completo" ou "full-difference") :



Geração e Controle de Marcação de Tempo

Transições de níveis

```
  borda                borda
de subida            de descida
(posedge)            (negedge)
              clock
          .`._______          1 _ nível lógico alto
        /  |       |`          _ 2/3
       /   |       | '         _ 1/3
  _____/   |       |  \_____ 0 _ nível lógico baixo
       |   |       |  |
  lag->|   |<-   ->|  |<-lag
(atraso)             (atraso)
```

1. *Clock* = gerador de sequências de alternâncias periódicas**

```
     __            __
  __|  |    ou    |  |__     = 1 pulso
  ts+tm =   T    = tm+ts     = 1 período (T) [s]
                               tm = tempo de marcação    (1)
                               ts = tempo de espaçamento (0)

  frequência = 1 / T [Hz]     (inverso do período)
```

```
   __    __    __
__|  |__|  |__|  |__  ...  = clock (sequência de pulsos)
```

2. Modelo para um gerador de pulsos (*Verilog*)**

```
   __    __    __
__|  |__|  |__|  |__  ...  = clock (sequência de pulsos)
12+12              =   T   = 24    (período)

        ^  ^   ^                          evento
        |  |   |__ em tempo de marcação @(clock)
        |  |______ na borda de descida  @(negedge clock)
        |_________ na borda de subida   @(posedge clock)
```

3. Sincronização de sinais**

```
   ___     ___     ___
__|   |___|   |___|   |__ ... = clock   (sequência de pulsos)
   _______
__|       |_____________ ... = pulso sincronizado com a borda de subida
                               em Verilog: @ (posedge clock)
       _______
______|       |__________ ... = pulso sincronizado com a borda de descida
                               em Verilog: @ (negedge clock)
     _______
____|       |___________ ... = pulso sincronizado com período de marcação
                               em Verilog: @ (clock)
```

Circuitos Combinacionais

Um circuito combinacional é aquele em que a(s) **saída(s) depende(m) de uma combinação das entradas**.

Circuitos Sequenciais

Um circuito sequencial, além de uma combinação das entradas, depende de uma combinação de outras variáveis que definem o estado em que o sistema se encontrava. Isto significa que um sistema deverá ter memória. Para passar ao próximo estado, precisará guardar informações sobre o estado atual.

Há dois tipos de circuitos sequenciais:

- **Assíncronos**: em que os estados podem mudar a qualquer instante.
- **Síncronos**: em que os estados mudam em instantes bem determinados

As mudanças de estados que ocorrerão em instantes determinados serão orientadas por um **sinal de temporização (*clock*)**.

Se ocorrerem as transições ocorrerem durante uma **variação de 0 para 1 (↑ - borda de subida)**, o sistema será dito de **nível alto**. Caso contrário, durante uma **variação de 1 para 0 (↓ - borda de descida)**, o sistema será dito de **nível baixo**.

As especificações de tempo para circuitos sequenciais também incluirão o tempo para a transição se estabilizar (*setup time*) e o tempo, após a transição, em que o sinal deve se mantiver constante (*hold time*).

Latches e Flip-Flops

Para entender *Latches* e *Flip-Flops* é necessário, primeiramente, entender como funcionam as máquinas de estados : 04_Maquinas_de_Estados.

Com o entendimento prévio, pode-se entender a dinâmica desses componentes que **servem para armazenar bit** (*bit-trap*).

Latches

Dependentes apenas dos sinais das entradas, "*preset*", "*clear*" ou composição das saídas (assíncrono).

- **SR (*Set-Reset*)** :

```
 ------
|      |
| S  Q |
|      |
| R  Q'|
|------|
  Pr Cl
```

- **D (*Data*)** :

```
 ------
|      |
| D  Q |
|      |
|    Q'|
```

```
      |______|
        Pr Cl
```

- **T (*Toggle*)** :

```
       ______
      |      |
      | T  Q |
      |      |
      |    Q'|
      |______|
        Pr Cl
```

- **JK (*Jump-Keep*)** :

```
       ______
      |      |
      | J  Q |
      |      |
      | K  Q'|
      |______|
        Pr Cl
```

Flip-Flops

Dependentes do "*clock*" e da construção, as mudanças ocorrem nas transições de bordas (ou de subida ou de descida) (assíncronos ou síncronos).

- **SR (*Set-Reset*)** :

```
Modelo de representação :
           ______
          |      |
          | S  Q |
      clk |>     |
          | R  Q'|
          |______|
            Pr Cl

Tabela característica :

      S  R  Qt+1
```

```
      0  0  Qt   - "hold"   - manter o sinal anterior (Qt+1=Qt)
      0  1  0    - "reset"  - levar para nível baixo  ( Q = 0 )
      1  0  1    - "set"    - levar para nível alto   ( Q = 1 )
      1  1  -    - "undefined"/"unused" (EVITAR !!!)

Mapa de Veitch-Karnaugh :

               Qt+1
      SR \ Qt  0  1
      00       0  1
      01       0  0
      11       x  x  <- considerando x=1 para simplificar
      10       1  1

      SoP = (1,4,5,6,7)
          = G1(4,5,6,7) + G2(1,5)
          = S           + R'.Qt

Equação característica (S.R=0) :

      Q    = S + R'.Q
       t+1           t

Diagrama de estados :

             S R
             1 0
            _____
      __   |     |   __
 S R |  \  |     v  /  | S R
 0 0 |   > 0     1 <   | 0 0
 0 1 |__/  ^     |  \__| 1 0
           |_____|
             0 1
             S R

Tabela de transições :

      Q -> Q     S R
       t    t+1

      0    0     0 X    (00="hold", 01="reset")
      0    1     1 0    (10="set"             )
      1    0     0 1    (01="reset"           )
      1    1     X 0    (00="hold", 10="set"  )
```

- **D (*Data*)**

```
Modelo de representação :
           ______
          |      |
          | D  Q |
      clk |>     |
          |    Q'|
          |______|
            Pr Cl

Tabela característica :

      D     Qt+1

      0     0    - "reset"   ( Q = 0 )
      1     1    - "set"     ( Q = 1 )

Mapa de Veitch-Karnaugh :

               Qt+1
      D \ Qt  0  1
      0       0  0
      1       1  1

      SoP (2,3) = D.Qt'+D.Qt
                = D.(Qt'+Qt) = D.1 = D

Equação característica :

      Q    = D
       t+1

Diagrama de estados :

               D
               1
            _____
      __   |     |   __
   D |  \  |     v  /  | D
   0 |   > 0     1 <   | 1
     |__/  ^     |  \__|
          |_____|
               0
               D
```

```
Tabela de transições :

      Q -> Q     D
       t    t+1

      0    0     0
      0    1     1
      1    0     0
      1    1     1
```

- **T (*Toggle*)**

```
Modelo de representação :

           ______
          |      |
          | T  Q |
      clk |>     |
          |    Q'|
          |______|
            Pr Cl

Tabela característica :

      T     Qt+1

      0     Qt   - "hold"    ( Qt+1 = Qt  )
      1     Qt'  - "toggle"  ( Qt+1 = Qt' )

Mapa de Veitch-Karnaugh   :

               Qt+1
      T  \ Qt  0  1
      0        0  1
      1        1  0
                              SoP (1,2) = T'.Qt + T.Qt' = T ^ Qt

Equação característica :

      Q    = T xor Q
       t+1          t

Diagrama de estados :
```

```
              T
              1
            _____
      __   |     |   __
   T |  \  |     v  /  | T
   0 |   > 0     1 <   | 0
     |__/  ^     |  \__|
          |_____|

              1
              T

Tabela de transições :

      Q -> Q     T
       t    t+1

      0    0     0
      0    1     1
      1    0     1
      1    1     0
```

- **JK (*Jump-Keep*)**

```
Modelo de representação :


           ______
          |      |
          | J  Q |
      clk |>     |
          | K  Q'|
          |______|
            Pr Cl

Tabela característica :

      J  K  Qt+1

      0  0  Qt   - "hold"    ( manter )
      0  1  0    - "reset"   ( Q = 0  )
      1  0  1    - "set"     ( Q = 1  )
      1  1  Qt'  - "toggle"  (alternar)

Mapa de Veitch-Karnaugh :
```

```
               Qt+1
      JK \ Qt  0  1
      00       0  1
      01       0  0
      11       1  0
      10       1  1

      SoP (1,4,5,6) = G1(4,6) + G2(1,5)
                    = J.Qt'   + K'.Qt
```

Equação característica :

```
      Q    = J.Q' + K'.Q
       t+1      t       t
```

Diagrama de estados :

```
             J=1

             J K
             1 0
             1 1
 J=0        _____        K=0
      __   |     |   __
 J K |  \  |     v  /  | J K
 0 0 |   > 0     1 <   | 0 0
 0 1 |__/  ^     |  \__| 1 0
           |_____|

             1 1
             0 1
             J K

             K=1
```

Diagrama de estados (simplificado) :

```
             J=1
            _____
      __   |     |   __
     |  \  |     v  /  |
 J=0 |   > 0     1 <   | K=0
     |__/  ^     |  \__|
           |_____|

             K=1
```

```
Tabela de transições :

      Q -> Q     J K
       t    t+1

      0    0     0 X    (00="hold" , 01="reset" )
      0    1     1 X    (10="set"  , 11="toggle")
      1    0     X 1    (01="reset", 11="toggle")
      1    1     X 0    (00="hold" , 10="set"   )
```